

Lab 3: Architectural Justification

Online Artisan Marketplace System

1. Architecture Selection

We chose **Microservices Architecture** for the **Online Artisan Marketplace System**.

The system is decomposed into five independently deployable components: *API Gateway*, *Order Manager*, *Payment Service*, *Catalog Service* and *Notification Service*. They communicate only through defined interfaces (*IOrderAPI*, *IStorefrontAPI*, *IPaymentProcessing*, *IInventory*, *INotification*), so each can be changed or replaced without affecting the others.

2. Reasons for the Choice

- **Independent scaling of catalog and checkout.** Shoppers browse far more than they buy, and NFR-001 requires high-resolution product images to load in under 500 ms. The Catalog Service can be scaled (and CDN-cached) on its own, without also scaling the lower-traffic Order Manager and Payment Service.
- **Fault isolation and multi-vendor workflows.** Checkout (UC-001) touches many concerns: stock validation, payment, split payout and notifications. Because Notification is a separate asynchronous service (*INotification*), an email outage does not block an order, while FR-004 still targets alerts within 10 seconds. Order Manager coordinates the steps and can roll back stock holds on payment failure (AF-1).

3. Security Advantage

Payment isolation. The Payment Service is the only component that handles card and banking data and the vendor payout ledger. Keeping it separate shrinks the attack surface and the compliance scope. It can enforce TLS on every call and AES-256 encryption at rest (NFR-002), and a breach in the Catalog or Notification services cannot expose financial credentials.

4. Performance Benefit

Targeted scaling and asynchronous processing. Read-heavy catalog traffic is served by the Catalog Service behind the API Gateway and a CDN, keeping browse latency low even at peak load. Slow, non-critical work such as sending notifications runs asynchronously through a message queue, so the shopper reaches the Order Confirmation page without waiting on it.

5. Trade-offs Acknowledged

Microservices add operational complexity, network latency between services, and data-consistency challenges (stock, ledger and order status must stay in step). We mitigate these with a single API Gateway entry point, TLS between services, and Order Manager acting as the coordinator that reverses stock holds if payment fails.